# Crusty Crab

# Online Seafood Solution

Functionality, Design, and Implementation


**Team 9**

**Tameez Lambat ST10462078**
**Zaara Salie ST10455456**
**Tapiwa Sango ST10276025**
**Cuan Oock ST10439216**
**Kaylen Samuels ST10443307**




# Topics covered


01 Table of contents

02 Problem being solved

03 Proposed System Overview

04 Front-end functionality

05 Back-end Architecture

06 Requirements

07 Non-Functional Requirements

08 Testing, Pipelines and Reliability

09 Automated Testing

10 Implementation Challenges

11 Known limits and roadmap

12 Thank you and Q&A

# Problem being solved

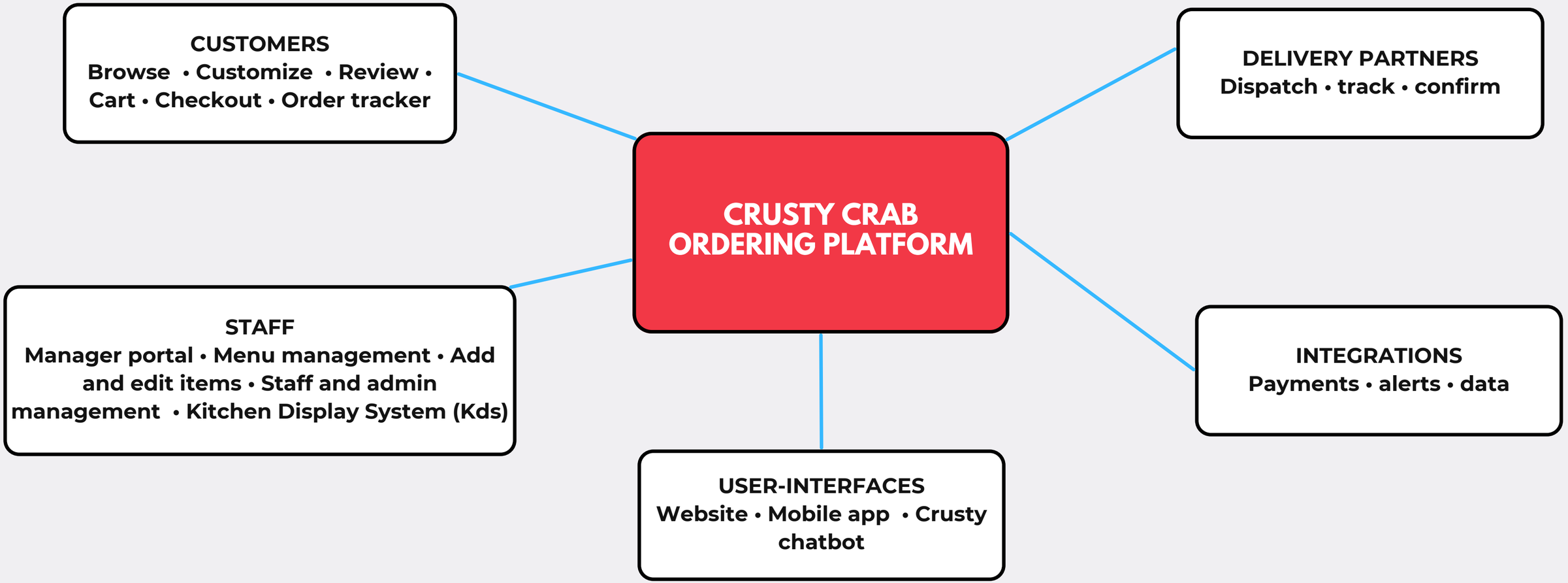


**Decrease the  reliability on WhatApp**

**WhatsApp-only ordering → website with no prices → "one owner answering every message**

- The current system of Crusty Crab is a *WhatsApp business platform* where customers browse the menu catalog and either message or call to place their orders. The owner answers each one manually.
- With a team of three, that creates *delays and room for human error*, and the *current website doesn't show prices or take orders*.
- Our goal is to let customers browse, customise, pay and track orders on their own, while giving our kitchen and manager the tools to run those orders smoothly.

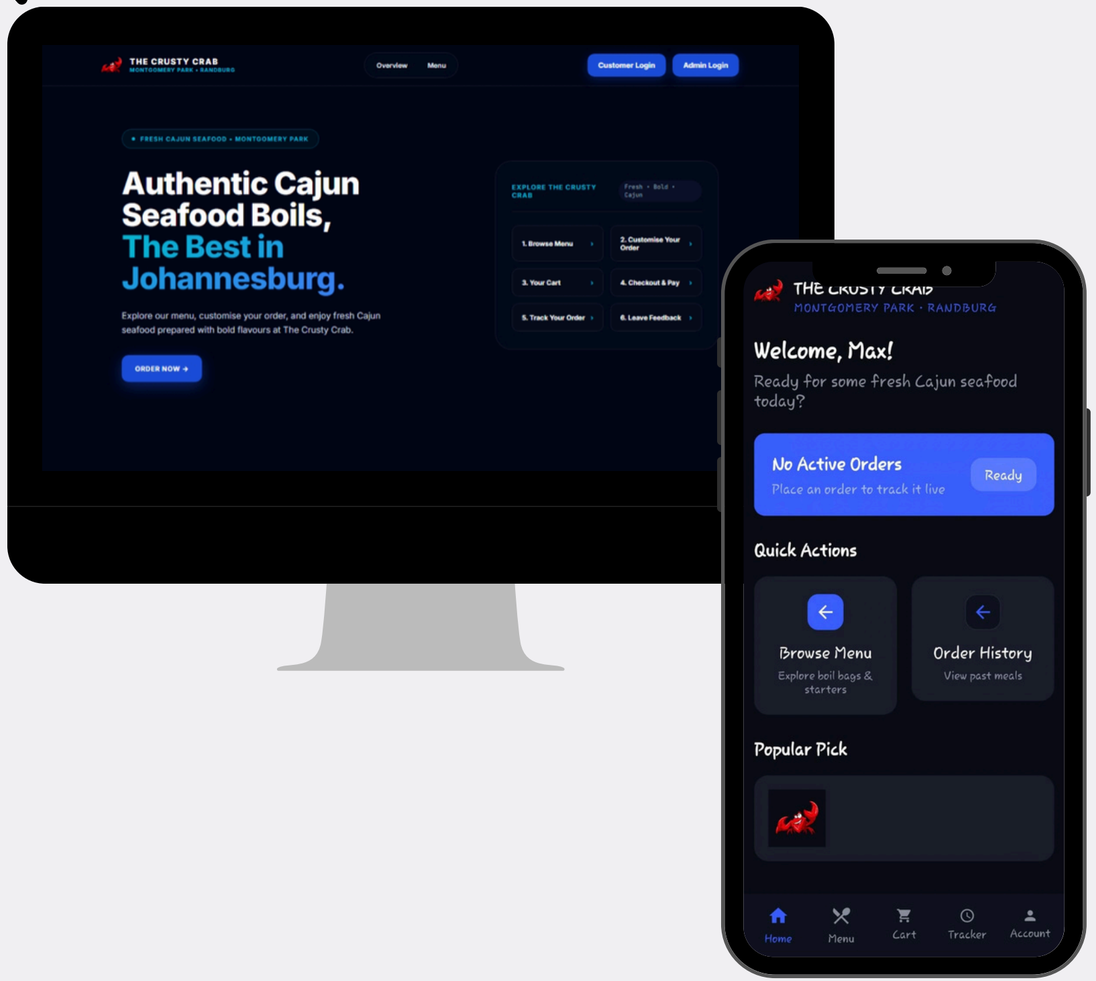
# Proposed System Overview




- Integration creates a seamless experience. Modular scaling keeps it fast as demand grows.

# Front-end functionality

**The front-end of the website was built using ASP.NET Core MVC (Razor views) and Kotlin was used for the mobile app.**



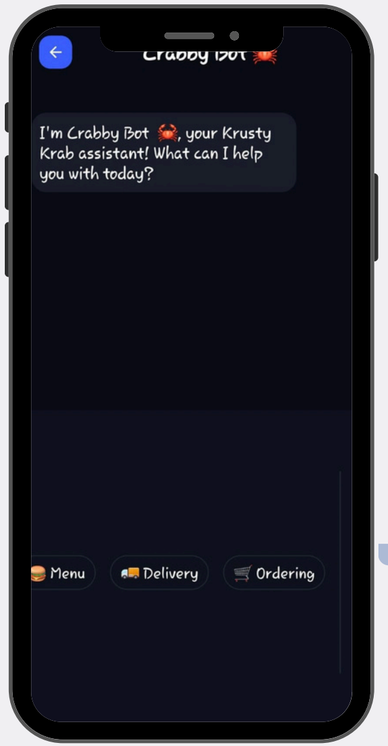

## Refurbished website

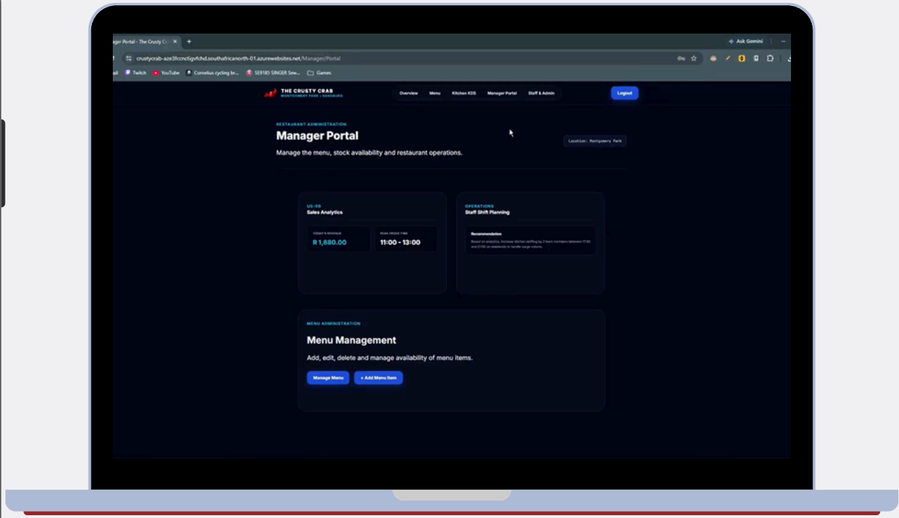
- Has more functionality, enabling customers to interact more with working buttons like to proceed to payment and review their order.
- Role-based access control (RBAC) is implemented to differentiate the customer dashboard from the admin login page.
- Customers can browse the menu, place orders and view their cart before checkout.
- Menu cards show image, description and price, with a Customize / Add button.

# Front-end functionality (Continued...)

## Brand new mobile application

- Each user has to register their details by providing their unique passwords and emails and use the same credentials in the web app as well.
- The customer's home page will display their active order, a popular pick and quick links to the menu and order history.
- Customers can also manage delivery addresses and saved payment methods, and track orders




## ChatBot assistant

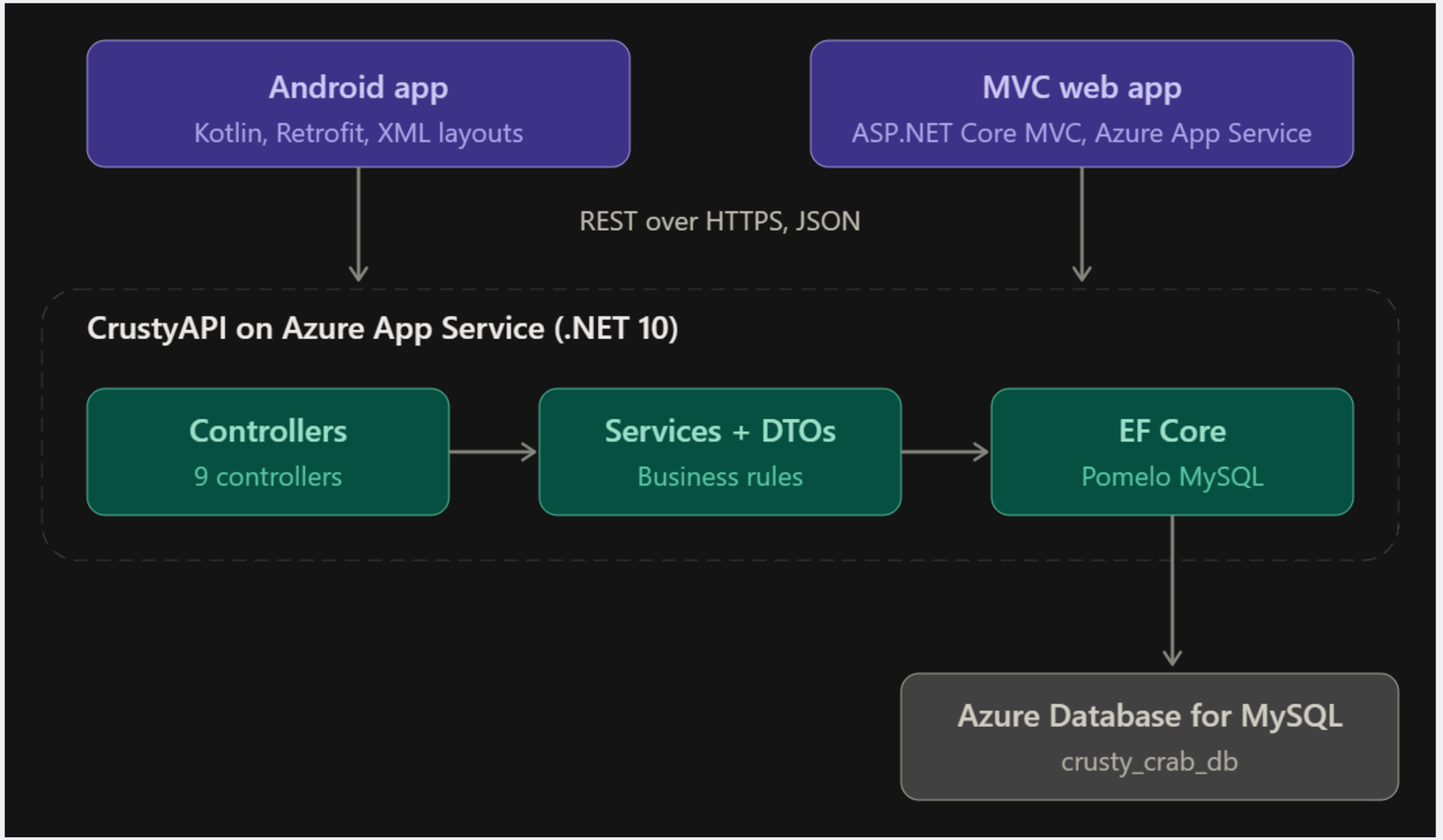
- Available to answer FAQ and guide customers through the ordering process.
- A list of options display simply by typing "start".

## Admin dashboard

- View insights of how well the business is running.
- Ability to see every order that comes in, can keep track of orders to work efficiently.
- Receive feedback to improve business processes.

**WEB APP DEMO**
**https://youtu.be/wsye5qpRHlw**
**MOBILE APP DEMO**
**https://youtube.com/shorts/sIX8pzTgMb4?feature=share**

# Back-end Architecture




- Kotlin · MySQL· encrypted data · event-driven status updates

# How the Back-end works

The back end is one shared REST API for the website and the Android app. The API is hosted on Azure App Service, which stores everything in a MySQL database.

It is split into three simple layers:



- controllers receive requests.
- services apply the business rules.
- the database stores the data.

It covers logins, customers and addresses, the menu, carts, orders, payments, staff and the Crabby Bot. Sensitive data stays private. Password hashes are never sent back to the apps.

Menu pictures are stored on the server, and managers can upload new ones from the website.

# Requirements

| Story | Status | Notes |
|---|---|---|
| US-01 Browse by category | Met | Web and Android |
| US-02 Customise boil bag | Met | Spice and add-ons; sauce choice next |
| US-03 Pickup or delivery | Met | Delivery fee added to total |
| US-04 Secure payment | Mostly met | Simulated flow; last 4 digits only, no real payment gate connection |
| US-05 Order tracking | Met | Updates on refresh; push next |
| US-06 Kitchen board | Mostly met | Oldest first; auto-refresh next |
| US-07 Mark order ready | Partly | One tap; SMS alert next |
| US-08 Out-of-stock toggle | Met | Blocked at cart and order |
| US-09 Sales insights | Partly | Today's revenue and peak hour |
| US-10 Help assistant | Met | FAQ assistant, not generative AI |

# Non-Functional requirements

### Usability

Responsive layouts + Consistent branding

### Performance

Lightweight bot + Request timeouts

### Scalability

Statleless API on Azure App Service

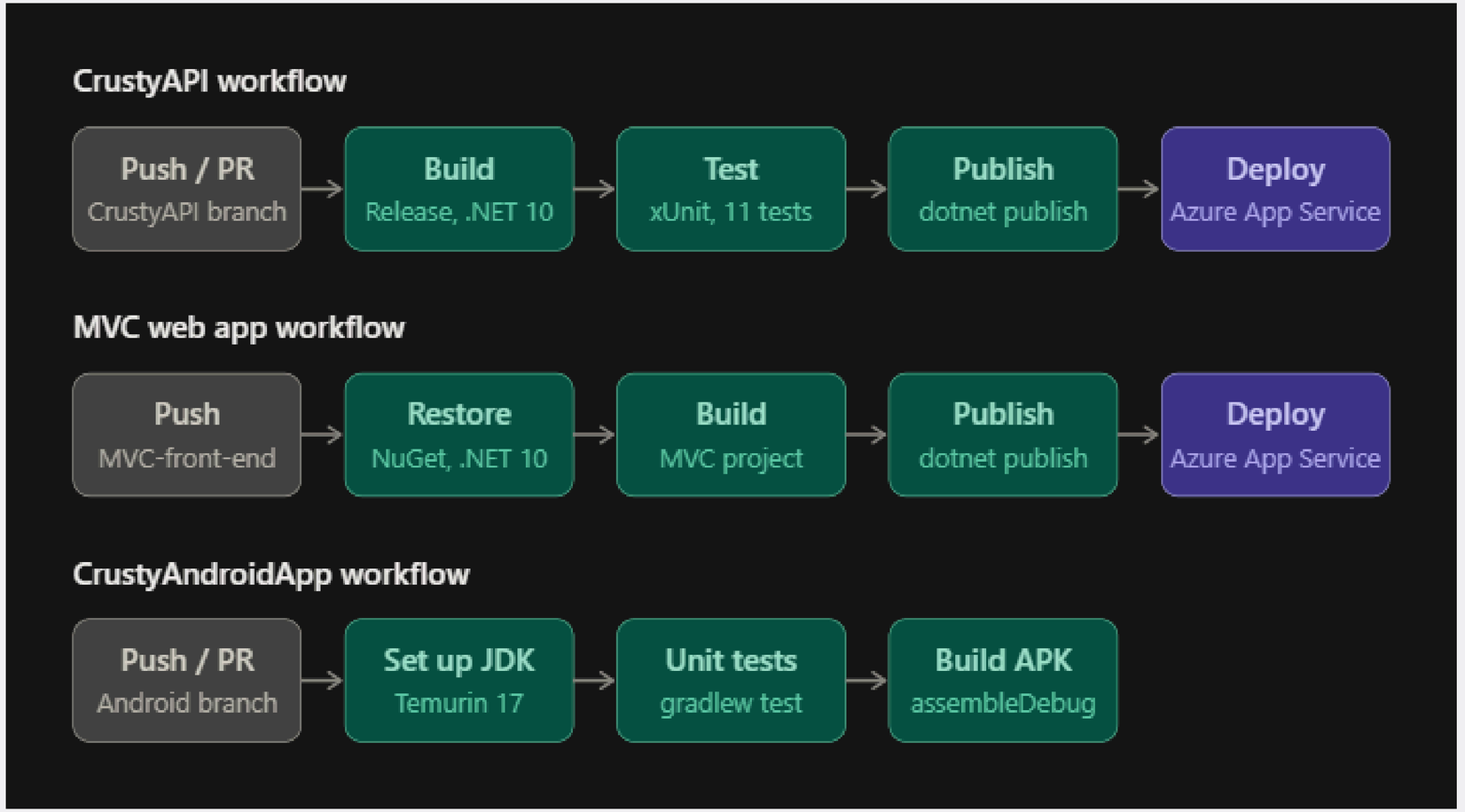
### Security

BCrypt hashing + Last 4 card storage + Role checks

### Availablity

Managed Azure App Service and MySQL

# Testing, Pipelines, and Reliability




**Reliability: request timeouts, friendly error messages, foreign-key constraints, and ownership checks on orders and addresses.**

# Testing, Pipelines, and Reliability (Continued...)

## Pipelines (GitHub Actions)

**API**: push or PR to the CrustyAPI branch → restore → build (Release) → test → publish → deploy to Azure using a publish-profile secret. Tests gate the deployment.

**Android**: push or PR to CrustyAndroidApp → JDK 17 → ./gradlew test → assembleDebug.
Using separate branches per project is a deliberate monorepo-style split you can mention.

## Testing (Automated Unit Testing)

**API**: 11 xUnit tests run against an in-memory database, so they need no server and finish in seconds.

- Login: valid login works, and a wrong password or unknown email is rejected.
- Cart: totals are calculated correctly, and invalid customers and unavailable items are refused.
- Orders: pending and preparing orders can be cancelled, completed ones can't, and one customer can't cancel another's order.

**Android**: 6 unit tests check that the order request is built correctly, including when the cart is empty and menu image addresses are resolved to the right web address.

# Implementation challenges

**01**

Debugging the android app. Unnecessary errors or warnings occured even though the code was accurate with no syntax errors.

**02**

Learning how to implement deployment pipeline in GitHub actions.

**05**

Issues with the virtual machine running applications which made development and testing difficult.

**04**

Some group members had more coding skills and experience than others.

# Known limits and roadmap

1. **API login tokens**

JWT and roles enforced on the API itself

2. **Live payments**

Connect a real payment gateway

3. **Live Updates**

Push notifications and kitchen auto-refresh

4. **Customer Feedback**

Collect it and show it to admins

5. **Safer Orders**

Database transaction around order creation

6. **Web Tests**

Automated tests in the web pipeline




# Thank You

## Questions & Discussion